Data Governance Copilot

Interview Question Bank

Topic 05 — Redis Caching Strategy

12 questions · 4 sections · @cached_node · TTL · SHA-256 · Fallback · Invalidation

ndational Core concepts | Intermediate Design decisions | Advanced Deep internals | Gotcha Real bugs / traps

---

Core Concepts — Redis & Caching Fundamentals

Q01 Foundational

Why did you add a caching layer in front of your LangGraph agent nodes? What problem does it solve?

Hint: Think about repeated identical queries in a multi-user system.

Key points to cover:

- LLM + external API calls (Databricks SQL, Collibra, pgvector) are expensive — latency 1-5s per node
- Business users often ask the same or similar questions (e.g. 'show me retention metrics for Q4')
- Without cache, every invocation re-hits Databricks SQL warehouse, Collibra REST, and pgvector HNSW search
- Cache allows near-instant response for repeat queries — sub-millisecond Redis GET vs 2-3s agent execution
- Also reduces LLM token spend — synthesizer re-uses cached agent_results without re-calling the LLM
- In ECS multi-task deployment, Redis is shared across all tasks — one task's cache benefits all

Q02 Foundational

Explain the @cached_node decorator. What does it wrap, what does it intercept, and what does it return on a cache hit?

Hint: Walk through the decorator execution path step by step.

Key points to cover:

- @cached_node(agent_name, ttl) is a decorator factory — returns a decorator that wraps the node function
- On call: first computes cache key via make_key(state) using SHA-256 of (query + data_products + time_range)
- Checks Redis (or in-memory fallback) for key — if hit, deserializes and returns cached AgentResult immediately
- If miss: executes the wrapped node function, serializes the AgentResult, stores with TTL, then returns result
- The wrapped function receives the full AgentState — decorator is transparent to the node's own logic
- Cache hit short-circuits the entire node including retry logic — retry only applies on actual execution

```
# Conceptual decorator structure def cached_node(agent_name, ttl): def decorator(fn): def wrapper(state): key = make_key(agent_name, state) hit = cache_get(key) # Redis GET if hit: return deserialize(hit) # skip node entirely result = fn(state) # run node cache_set(key, serialize(result), ttl=ttl) return result return wrapper return decorator
```

Q03 Intermediate

How is the cache key constructed? Why SHA-256 instead of a plain string concatenation?

Hint: Think about key length, collision resistance, and what inputs vary between queries.

Key points to cover:

- Cache key = SHA-256(query + sorted(data_products) + time_range) — hexdigest gives 64-char fixed-length string
- Plain concatenation risk: 'sales_q1' + 'revenue' collides with 'sales' + '_q1revenue' — SHA-256 avoids this
- Redis key length: SHA-256 is always 64 chars vs potentially unbounded query strings (up to 2000 chars)
- data_products must be sorted before hashing — [retention, bookings] and [bookings, retention] are same query
- agent_name is prefixed to the key to namespace across agents: 'information:abc123...', 'knowledge:def456...'
- SHA-256 is one-way — no PII leakage in Redis key if query contains sensitive terms

```
import hashlib, json def make_key(agent_name, state): payload = json.dumps({ 'q': state.get('query', ''), 'dp': sorted(state.get('data_products', [])), 'tr': state.get('time_range', ''), }, sort_keys=True) digest = hashlib.sha256(payload.encode()).hexdigest() return f'{agent_name}:{digest}'
```

TTL Strategy Per Agent

Q04 Intermediate

Your three cached agents have different TTLs: 30 min, 2 hours, 1 hour. Walk through the reasoning for each.

Hint: TTL should reflect how fast the underlying data changes.

Key points to cover:

- information_node (30 min / 1800s): hits Databricks SQL warehouse — business metrics update frequently (hourly pipelines), stale data beyond 30 min risks misleading analysis
- knowledge_node (2 hrs / 7200s): hits pgvector RAG — governance documents, policies, wikis change rarely (days/weeks), longer cache is safe and saves expensive HNSW vector searches
- metadata_node (1 hr / 3600s): hits Collibra REST API — data catalog metadata updates daily via collibra_sync_dag at 06:00 UTC, 1 hr is conservative and appropriate
- capacity_node is NOT cached — Jira ticket states change in real-time (open/in-progress/closed); stale incident data is dangerous
- auto_ticket_node is NOT cached — write operations must never be cached (idempotency risk)
- synthesizer_node is NOT cached — its output depends on the combination of all agent results, which is already cached individually

---

Q05 Advanced

What happens if metadata changes in Collibra mid-TTL? How would you handle cache invalidation for that scenario?

Hint: This is about proactive vs reactive invalidation.

Key points to cover:

- Current behaviour: stale metadata served until TTL expires (up to 1 hour) — acceptable for most governance use cases
- Reactive fix: call invalidate_pattern(client, 'metadata:*') after the collibra_sync_dag completes at 06:00 UTC
- Add an Airflow task at the end of collibra_sync_dag that POSTs to /ingest or calls the MCP invalidate_cache tool
- Proactive fix: Collibra webhooks on asset updates → FastAPI endpoint → invalidate specific key for that asset
- Selective invalidation: store asset_id in cache key — invalidate only affected keys, not all metadata cache
- In-memory fallback cannot be invalidated across ECS tasks — another reason Redis shared cache is critical in prod

---

Q06 Advanced

If TTL is set to 1800 seconds and 50 ECS tasks all start simultaneously, what cache problem can occur and how do you fix it?

Hint: This is the cache stampede / thundering herd problem.

Key points to cover:

- Cache stampede: all 50 tasks get a cache miss simultaneously (cold start or TTL expiry) — all execute the expensive agent in parallel
- Results in 50 concurrent Databricks SQL queries / Collibra API calls — can overwhelm downstream services
- Fix 1: Cache locking — use Redis SET NX (set if not exists) as a distributed lock; only one task executes, others wait then read
- Fix 2: Probabilistic early expiration (PER) — randomly re-compute cache before TTL expires to stagger refreshes
- Fix 3: Jitter on TTL — instead of exact 1800s, use random.randint(1700, 1900) to spread expiry across tasks
- Fix 4: Background refresh — a dedicated worker pre-warms cache before TTL expires for common queries
- Your current implementation doesn't handle stampede — valid gap to acknowledge and explain mitigations

```
# TTL jitter pattern import random def cached_node_with_jitter(agent_name, ttl, jitter=60): def decorator(fn): def wrapper(state): key = make_key(agent_name, state) hit = cache_get(key) if hit: return deserialize(hit) result = fn(state) actual_ttl = ttl + random.randint(-jitter, jitter) cache_set(key, serialize(result), ttl=actual_ttl) return result return wrapper return decorator
```

In-Memory Fallback & Redis Connection

Q07 Intermediate

Describe the in-memory fallback mechanism. When does it activate and what are its limitations in production?

Hint: Think about what 'fallback' means for a distributed system.

Key points to cover:

- At module import time, cache.py probes Redis with a 1-second timeout — if unreachable, sets _redis_client = None
- All cache operations check: if _redis_client is None → use _in_memory dict; else use Redis
- _in_memory is a plain Python dict — no TTL enforcement natively (must track expiry timestamps separately)
- Critical limitation: in-memory is per-process — 50 ECS tasks each have their own dict, zero sharing
- Same query from task-1 and task-2 both execute the agent — cache provides no benefit across tasks
- In-memory fallback is valid for dev/CI (single process, REDIS_ENABLED=false) but must NEVER be the prod strategy
- Prod safeguard: health check endpoint /agents/status exposes Redis connectivity — alert if Redis is down

---

Q08 Intermediate

The rate limiter in middleware.py also uses Redis with a memory:// fallback. How does this interact with your cache Redis client — are they the same connection?

Hint: Think about connection pooling and configuration isolation.

Key points to cover:

- They are separate Redis clients — cache.py creates its own client; slowapi Limiter creates another via storage_uri
- Both point to the same Redis instance (REDIS_HOST/PORT) but are independent connections
- This means two Redis connection pools in the same process — could be optimised by sharing a pool
- Benefit of separation: cache failure doesn't affect rate limiting and vice versa — independent fallback logic
- middleware.py probes Redis at import time (same pattern) — memory:// fallback means per-process rate limits
- In prod: both must reach Redis for correct behaviour — shared ElastiCache cluster handles both concerns
- Improvement: use a single redis.ConnectionPool shared across cache and limiter clients

---

invalidate_pattern & Cache Management

Q09 Intermediate

Explain the invalidate_pattern function. How does it work and what does it return?

Hint: Think about Redis SCAN vs KEYS and why that matters.

Key points to cover:

- invalidate_pattern(client, pattern) deletes all Redis keys matching a glob pattern (e.g. 'information:*')
- Uses Redis SCAN cursor iteration — non-blocking, safe for production (KEYS command blocks Redis event loop)
- Collects matching keys in batches, then pipelines DEL commands — returns count of deleted keys
- The MCP tool invalidate_cache exposes this: agent_name='all' → deletes all agent namespaces; specific name → one namespace
- Also resets the _in_memory fallback dict for the matching agent — important for dev consistency
- Return value (int) lets callers verify: 0 means nothing was cached for that agent (useful for debugging)

```
# Conceptual implementation def invalidate_pattern(client, pattern) -> int: count = 0 cursor = 0 while True: cursor, keys = client.scan(cursor, match=pattern, count=100) if keys: client.delete(*keys) count += len(keys) if cursor == 0: break return count
```

Q10 Advanced

A user runs a query, gets a cached result, then updates a governance rule. The next query still returns the old cached result. How do you fix this architecturally?

Hint: This is cache coherence — writes must invalidate related reads.

Key points to cover:

- Root cause: write operations (rule_node, metadata writes) don't invalidate read caches for affected agents
- Fix 1: Write-through invalidation — in rule_node after a successful write, call invalidate_pattern('knowledge:*') and invalidate_pattern('metadata:*')
- Fix 2: Event-driven invalidation — publish a Redis Pub/Sub event on every write; a subscriber deletes affected keys
- Fix 3: Tag-based caching — tag cache entries with entity IDs (rule_id, asset_id); on write, delete by tag
- Fix 4: Short TTL on write-sensitive agents — reduce knowledge_node TTL to 5 min if rules change frequently
- Your current design has this gap — a valid interview point to raise proactively and explain the tradeoff (simplicity vs coherence)
- Pragmatic answer: governance rules change rarely in practice — TTL expiry is acceptable; add explicit invalidation for the MCP tool

---

Q11 Gotcha

What happens if you cache an AgentResult that contains a Python exception object or a failed result? Could this cause 'negative caching'?

Hint: Think about what AgentResult(success=False) looks like when serialized.

Key points to cover:

- Negative caching: caching a failure result — subsequent requests get the cached failure even if the underlying service has recovered
- AgentResult has a success: bool field — if success=False, the result still gets cached with the full TTL
- Impact: if Databricks SQL is down for 10 seconds, the failure gets cached for 1800 seconds — 30 min of false errors
- Fix: only cache AgentResult where result.success is True — failures should not be written to cache
- Add check before cache_set: if not result.success: return result (skip caching)
- Also: never cache AgentResult containing exception objects — they may not be JSON-serializable
- Your current implementation should be checked for this — it is a common caching bug in agent systems

```
# Correct pattern: guard before caching def wrapper(state): key = make_key(agent_name, state) hit = cache_get(key) if hit: return deserialize(hit) result = fn(state) if result.success: # only cache successes cache_set(key, serialize(result), ttl=ttl) return result
```

Q12 Design

In production on AWS, your Redis is ElastiCache. What ElastiCache configuration decisions matter for this caching use case?

Hint: Think about eviction, persistence, cluster mode, and TLS.

Key points to cover:

- Eviction policy: allkeys-lru (evict least-recently-used when memory full) — correct for a cache-only workload; never noeviction
- Persistence: disable RDB/AOF snapshots — cache doesn't need durability; persistence adds I/O overhead
- Cluster mode: single-node with read replica is sufficient for this workload — cluster mode adds complexity without clear benefit at this scale
- TLS in transit: enable for Neon/prod — set REDIS_SSL=true; ElastiCache supports TLS on port 6380
- AUTH token: enable Redis AUTH on ElastiCache; store token in AWS Secrets Manager, inject via env var
- Instance sizing: cache.t3.micro sufficient for dev; cache.r6g.large for prod (memory-optimised, Graviton)
- Monitoring: CloudWatch ElastiCache metrics — CacheHitRate, Evictions, CurrConnections, NetworkBytesIn/Out

---